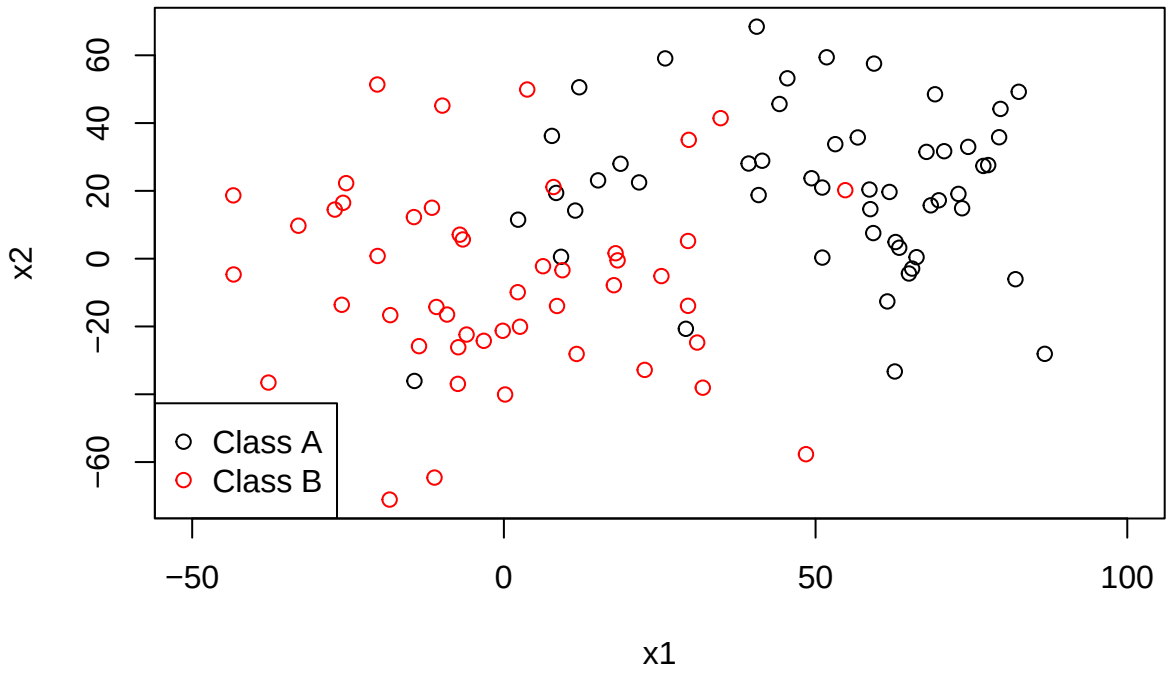
Classification Trees

Classification trees (CART)

Classification trees partition the data into a series of hierarchical branches, where each branch point corresponds to one of the predictors. The first branch will be the variable that best predicts the value of the response. The second branch point will best predict the response in the groups produced by the first branch. It continues this was until the algorithm stops.

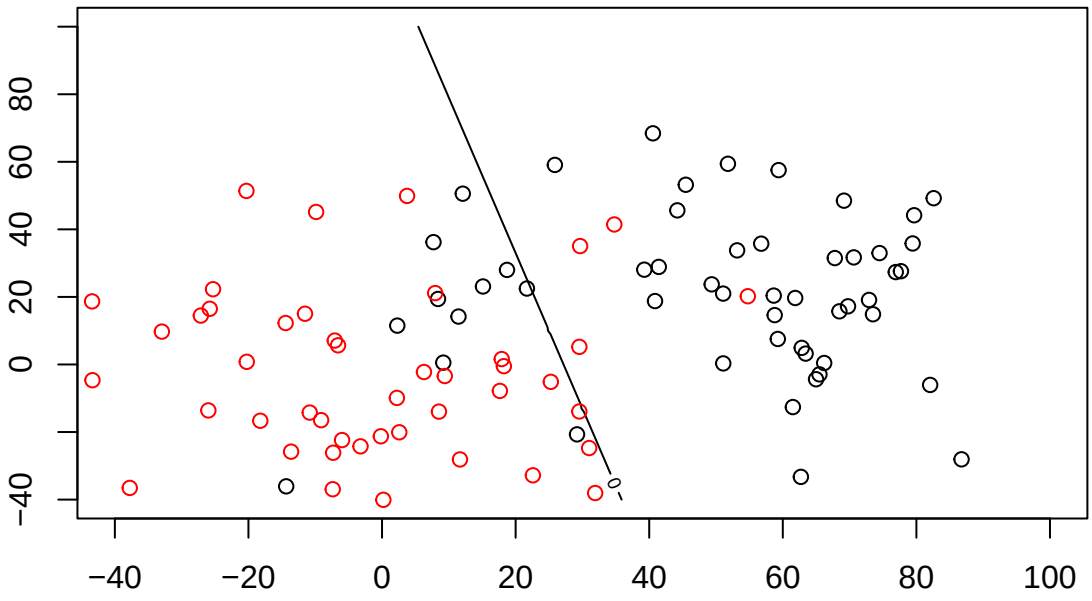
We will start first with two simple simulated examples that demonstrate the usefulness of classification trees. This data has two predictors `x1` and `x2`. The means for the two groups for `x1` are separated by 50 and those for `x2` are separated by 20. There is an SD of 15 in both dimensions.

```
library(MASS)
n=100 #sample size
muA=c(50,20) #mean for class A
muB=c(0,0) # mean for class B
sd1=25 #standard deviation
sd2=25
cor0=0.2
sigma0=matrix(c(sd1^2,cor0*sd1*sd2,cor0*sd1*sd2,sd2^2),nrow=2)

piA=0.5  #prob of class A
nA=rbinom(1,n,piA) # number of class A
nB=n-nA

xA=mvrnorm(nA,mu=muA,Sigma=sigma0)
xB=mvrnorm(nB,mu=muB,Sigma=sigma0)
x=rbind(xA,xB)

plot(x,col=c(rep("black",nA),rep("red",nB)), xlab="x1",ylab="x2",xlim=c(-50,100))
legend("bottomleft",pch=1,c("Class A","Class B"),col=c("black","red"))
```



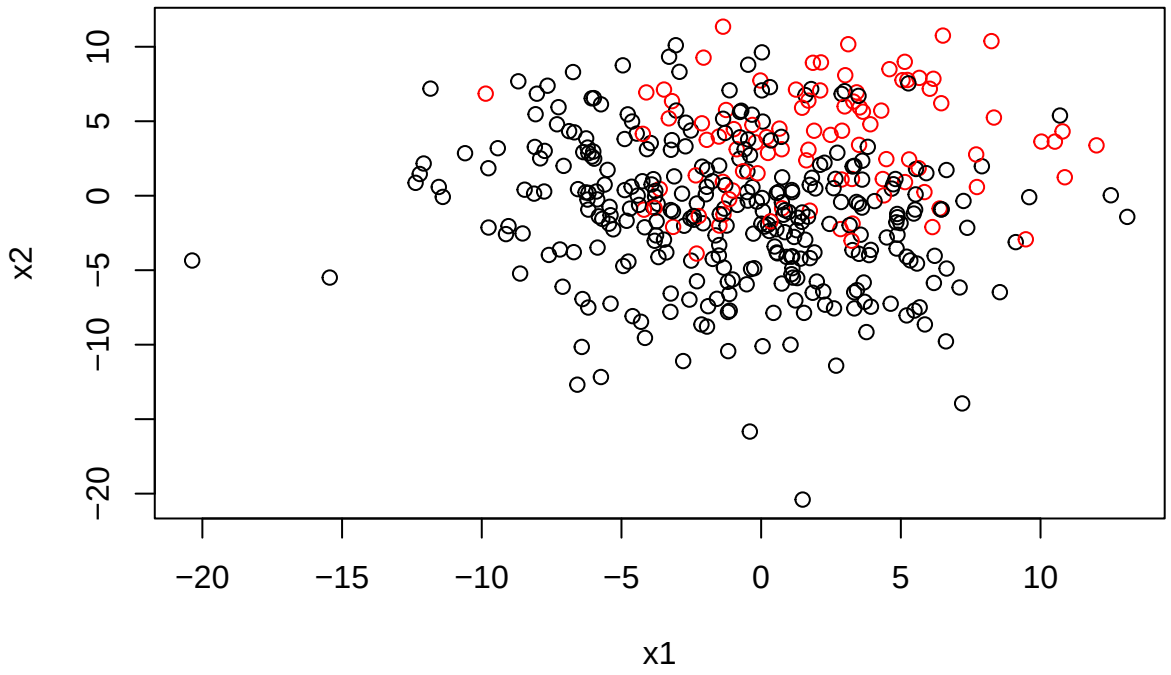
The code below shows the Bayes decision boundary, which is like LDA but with the true parameter values rather than estimates.Students in STAT 8331 will recognize the code. We will not worry about details here.

```
deltak<-function(x,Sig,muk,pik)
{
  Siginv=solve(Sig)

return(t(x)%*%Siginv%*%muk-0.5*t(muk)%*%Siginv%*%muk+log(pik))

}



x1vals=seq(from=-40,to=100,by=1)
x2vals=x1vals

deltaD=matrix(nrow=length(x1vals),ncol=length(x2vals))
i=0
for(x1 in x1vals)
{ i=i+1
  j=0
  for(x2 in x2vals)
{ j=j+1
  deltaD[i,j]=deltak(c(x1,x2),sigma0,muA,piA)-deltak(c(x1,x2),sigma0,muB,1-piA)
}}


contour(x1vals,x2vals,deltaD, levels=0)
points(x,col=c(rep("black",nA),rep("red",nB)))
```



We see that a linear boundary does well at dividing this data. Now consider another simulated data set:

```
n <- 400
mu1=0
sd1=5
sd2=5
mu2=0
sd0=3
X1Abound=0
X2Abound=0

x1 <- rnorm(n,mean=mu1,sd=sd1)
x2 <- rnorm(n,mean=mu2,sd=sd2)
Y=rep("A",n)

Y[((x1+rnorm(n,sd=2*sd0))>X1Abound)&((x2+rnorm(n,sd=sd0))>X2Abound)]="B"

plot(x1,x2,col=ifelse(Y=="A","black","red"))
```




```
dat1=data.frame(x1,x2,Y)
```

This data involves an interaction between the variables `x1` and `x2`.

There is no straight line that we can draw that will separate the classes well. The problem is the interaction. Because we simulated the data, we know that the best decision rule would be

predicted class is A if x1>0 and x2>0. Predicted class is B otherwise

For this data, we need a classification method that can produce decisions rules of this nature. This brings us to **classification trees**.

Classification trees (CART)

Classification trees partition the data into a series of hierarchical branches, where each branch point corresponds to one of the predictors. The first branch will be the variable that best predicts the value of the response. The second branch point will best predict the response in the groups produced by the first branch. It continues this was until the algorithm stops.

Consider the `Carseats` data. This is a simulated data set containing sales of car seats at 400 different stores. We wish to find out which variables are most important in determining whether a Carseat will have high sales.

We will first turn the `Sales` variable into a binary response. We consider sales for a car seat `High` if the sales are at least 8,000. We create a binary variable `High` and add it into the data frame:

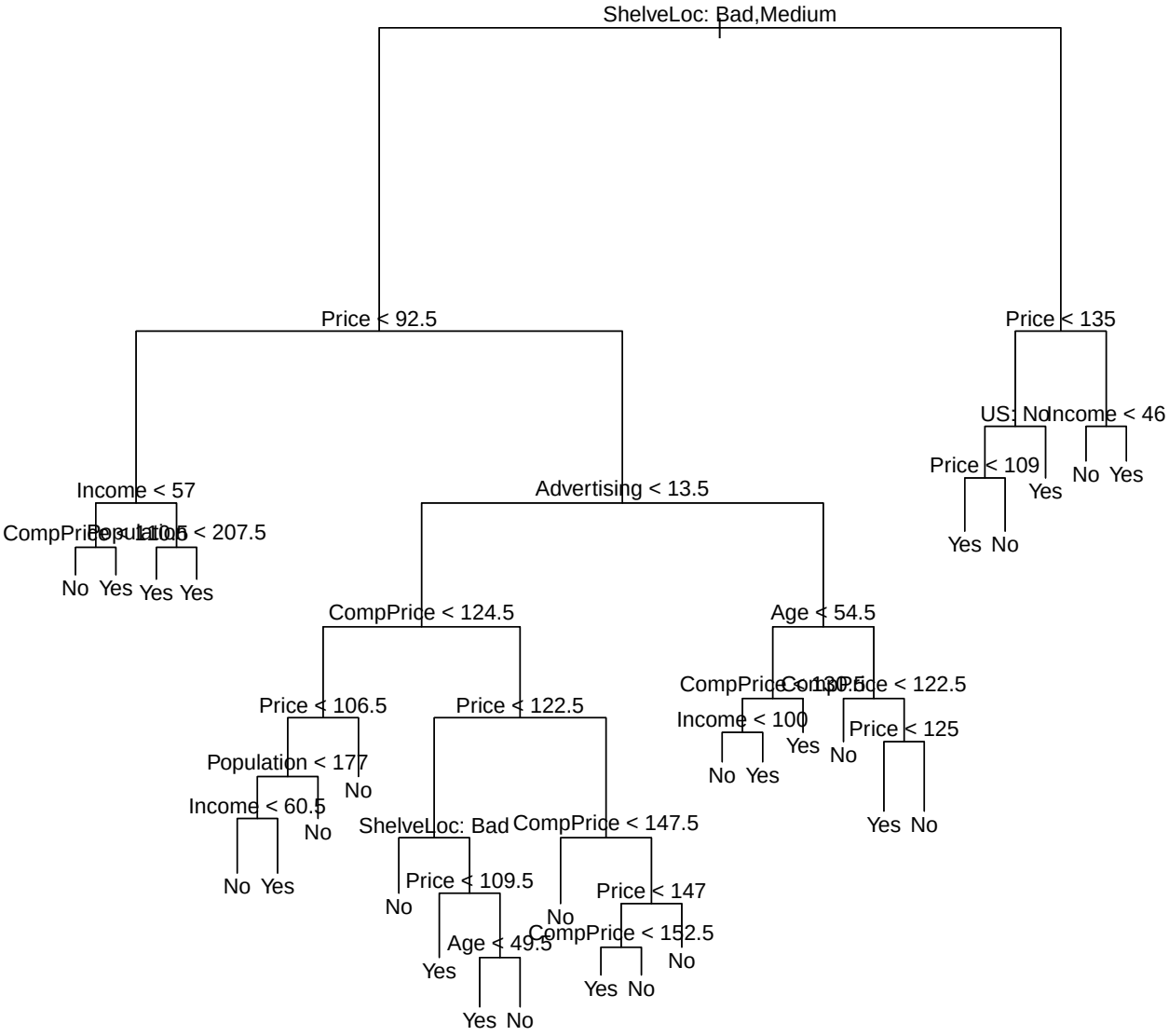
```
library(tree)
library(ISLR)
attach(Carseats)

High=as.factor(ifelse(Sales<=8,"No","Yes")) #recoding Sales as a binary response variable
Carseats=data.frame(Carseats,High)
head(Carseats)
```

```
##   Sales CompPrice Income Advertising Population Price ShelveLoc Age Education
## 1  9.50       138     73          11        276   120       Bad  42        17
## 2 11.22       111     48          16        260    83      Good  65        10
## 3 10.06       113     35          10        269    80    Medium  59        12
## 4  7.40       117    100           4        466    97    Medium  55        14
## 5  4.15       141     64           3        340   128       Bad  38        13
## 6 10.81       124    113          13        501    72       Bad  78        16
##   Urban  US High
## 1   Yes Yes  Yes
## 2   Yes Yes  Yes
## 3   Yes Yes  Yes
## 4   Yes Yes   No
## 5   Yes  No   No
## 6    No Yes  Yes
```

The `tree` command in the `tree` library fits a classification tree:

Example: Fitting Classification Trees: 10 predictors, Sales: response, continuous

```
tree.carseats=tree(High~.-Sales,Carseats) #exclude Sales

plot(tree.carseats) #display the tree structure
text(tree.carseats,pretty=0) #display the node labels, "pretty=0" includes the category names for any q
```



The most important indicator of Sales appears to be shelving location, since the first branch differentiates Good locations from Bad and Medium locations.

Within shelving location, Price is the next most important variable. For Good shelving locations, the tree partitions by whether the price is greater or less than \$135. In medium or bad locations, the partition is on whether price is greater or less than \$92.50.

This shows full details of the tree:

```
tree.carseats
```

```
## node), split, n, deviance, yval, (yprob)
##       * denotes terminal node
##
##   1) root 400 541.500 No ( 0.59000 0.41000 )
##     2) ShelveLoc: Bad,Medium 315 390.600 No ( 0.68889 0.31111 )
##       4) Price < 92.5 46  56.530 Yes ( 0.30435 0.69565 )
##         8) Income < 57 10  12.220 No ( 0.70000 0.30000 )
##          16) CompPrice < 110.5 5   0.000 No ( 1.00000 0.00000 ) *
##          17) CompPrice > 110.5 5   6.730 Yes ( 0.40000 0.60000 ) *
```

```
##         9) Income > 57 36  35.470 Yes ( 0.19444 0.80556 )
##          18) Population < 207.5 16  21.170 Yes ( 0.37500 0.62500 ) *
##          19) Population > 207.5 20   7.941 Yes ( 0.05000 0.95000 ) *
##       5) Price > 92.5 269 299.800 No ( 0.75465 0.24535 )
##        10) Advertising < 13.5 224 213.200 No ( 0.81696 0.18304 )
##          20) CompPrice < 124.5 96  44.890 No ( 0.93750 0.06250 )
##            40) Price < 106.5 38  33.150 No ( 0.84211 0.15789 )
##              80) Population < 177 12  16.300 No ( 0.58333 0.41667 )
##               160) Income < 60.5 6   0.000 No ( 1.00000 0.00000 ) *
##               161) Income > 60.5 6   5.407 Yes ( 0.16667 0.83333 ) *
##              81) Population > 177 26   8.477 No ( 0.96154 0.03846 ) *
##            41) Price > 106.5 58   0.000 No ( 1.00000 0.00000 ) *
##          21) CompPrice > 124.5 128 150.200 No ( 0.72656 0.27344 )
##            42) Price < 122.5 51  70.680 Yes ( 0.49020 0.50980 )
##              84) ShelveLoc: Bad 11   6.702 No ( 0.90909 0.09091 ) *
##              85) ShelveLoc: Medium 40  52.930 Yes ( 0.37500 0.62500 )
##               170) Price < 109.5 16   7.481 Yes ( 0.06250 0.93750 ) *
##               171) Price > 109.5 24  32.600 No ( 0.58333 0.41667 )
##                 342) Age < 49.5 13  16.050 Yes ( 0.30769 0.69231 ) *
##                 343) Age > 49.5 11   6.702 No ( 0.90909 0.09091 ) *
##            43) Price > 122.5 77  55.540 No ( 0.88312 0.11688 )
##              86) CompPrice < 147.5 58  17.400 No ( 0.96552 0.03448 ) *
##              87) CompPrice > 147.5 19  25.010 No ( 0.63158 0.36842 )
##               174) Price < 147 12  16.300 Yes ( 0.41667 0.58333 )
##                 348) CompPrice < 152.5 7   5.742 Yes ( 0.14286 0.85714 ) *
##                 349) CompPrice > 152.5 5   5.004 No ( 0.80000 0.20000 ) *
##               175) Price > 147 7   0.000 No ( 1.00000 0.00000 ) *
##        11) Advertising > 13.5 45  61.830 Yes ( 0.44444 0.55556 )
##          22) Age < 54.5 25  25.020 Yes ( 0.20000 0.80000 )
##            44) CompPrice < 130.5 14  18.250 Yes ( 0.35714 0.64286 )
##              88) Income < 100 9  12.370 No ( 0.55556 0.44444 ) *
##              89) Income > 100 5   0.000 Yes ( 0.00000 1.00000 ) *
##            45) CompPrice > 130.5 11   0.000 Yes ( 0.00000 1.00000 ) *
##          23) Age > 54.5 20  22.490 No ( 0.75000 0.25000 )
##            46) CompPrice < 122.5 10   0.000 No ( 1.00000 0.00000 ) *
##            47) CompPrice > 122.5 10  13.860 No ( 0.50000 0.50000 )
##              94) Price < 125 5   0.000 Yes ( 0.00000 1.00000 ) *
##              95) Price > 125 5   0.000 No ( 1.00000 0.00000 ) *
##     3) ShelveLoc: Good 85  90.330 Yes ( 0.22353 0.77647 )
##       6) Price < 135 68  49.260 Yes ( 0.11765 0.88235 )
##        12) US: No 17  22.070 Yes ( 0.35294 0.64706 )
##          24) Price < 109 8   0.000 Yes ( 0.00000 1.00000 ) *
##          25) Price > 109 9  11.460 No ( 0.66667 0.33333 ) *
##        13) US: Yes 51  16.880 Yes ( 0.03922 0.96078 ) *
##       7) Price > 135 17  22.070 No ( 0.64706 0.35294 )
##        14) Income < 46 6   0.000 No ( 1.00000 0.00000 ) *
##        15) Income > 46 11  15.160 Yes ( 0.45455 0.54545 ) *
```

This displays the split criterion (e.g. Price<92.5), the number of observations in that branch, the deviance, the overall prediction for the branch (Yes or No), and the fraction of observations in that branch that take on values of Yes and No.

The tree has a 9% error rate:

```
summary(tree.carseats) #9% error rate
```

```
##
## Classification tree:
## tree(formula = High ~ . - Sales, data = Carseats)
## Variables actually used in tree construction:
## [1] "ShelveLoc"   "Price"       "Income"      "CompPrice"   "Population"
## [6] "Advertising" "Age"         "US"
## Number of terminal nodes:  27
## Residual mean deviance:  0.4575 = 170.7 / 373
## Misclassification error rate: 0.09 = 36 / 400
```

Using training/testing data sets

Now we randomly split the data into testing and training data.

We build the classification tree using only the test data (`subset=test`)

```
set.seed(2)
train=sample(1:nrow(Carseats), 200)
Carseats.test=Carseats[-train,]
High.test=High[-train]
tree.carseats=tree(High~.-Sales,Carseats,subset=train)
```

Now we test the tree on the test data:

```
tree.pred=predict(tree.carseats,Carseats.test,type="class")
table(tree.pred,High.test)
```

```
##          High.test
## tree.pred  No Yes
##       No  104  33
##       Yes  13  50
```

```
mean(tree.pred==High.test)
```

```
## [1] 0.77
```

Pruning

The basic method above will add all of the variables into the tree. However, this will often produce overfitting. The function `cv.tree()` performs cross-validation in order to determine the optimal level of tree complexity; cost complexity pruning is used in order to select a sequence of trees for consideration. We use the argument `FUN=prune.misclass` in order to indicate that we want the classification error rate to guide the cross-validation and pruning process, rather than the default for the `cv.tree()` function, which is deviance. The `cv.tree()` function reports the number of terminal nodes of each tree considered (size) as well as the corresponding error rate and the value of the cost-complexity parameter used.

```
set.seed(3)
cv.carseats=cv.tree(tree.carseats,FUN=prune.misclass)
names(cv.carseats)
```

```
## [1] "size"   "dev"    "k"      "method"
```

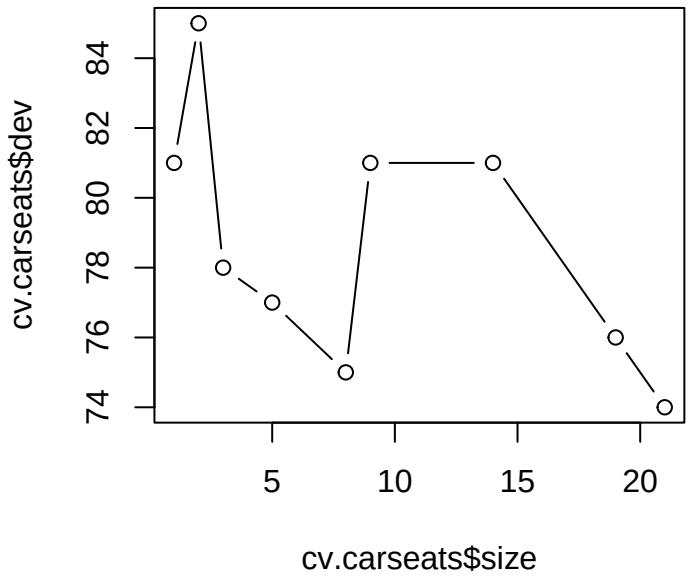
```
cv.carseats
```

```
## $size
## [1] 21 19 14  9  8  5  3  2  1
##
## $dev
## [1] 74 76 81 81 75 77 78 85 81
##
## $k
## [1] -Inf  0.0  1.0  1.4  2.0  3.0  4.0  9.0 18.0
##
## $method
## [1] "misclass"
##
## attr(,"class")
## [1] "prune"         "tree.sequence"
```

Despite the name, `dev` corresponds to the cross-validation error rate in this instance.

Here is a plot of error rate versus the number of terminal nodes:

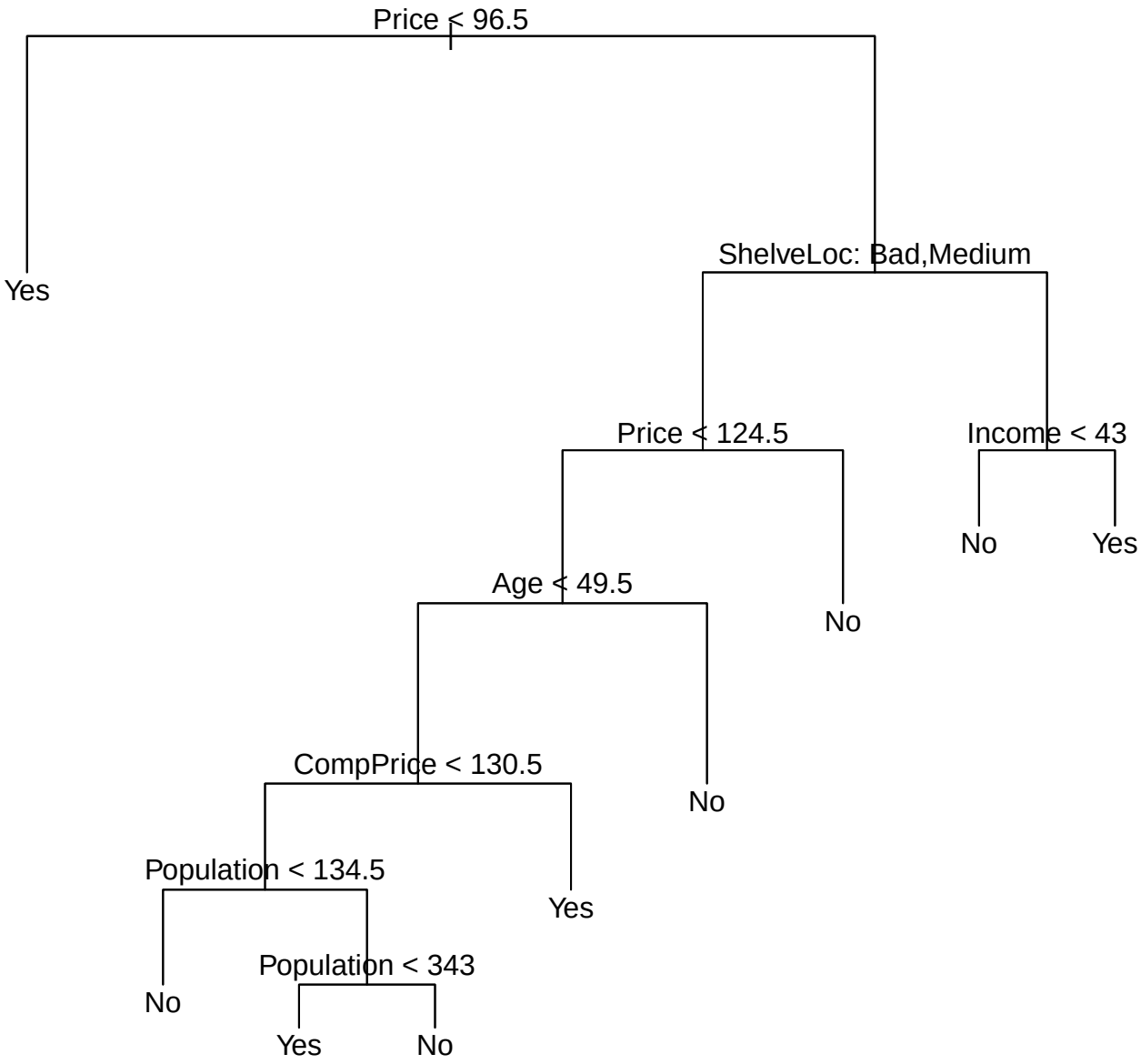
```
plot(cv.carseats$size,cv.carseats$dev,type="b")
```



The tree with 9 terminal nodes results in the lowest cross-validation error rate.

Now, we prune the classification tree to 9 nodes:

```
prune.carseats=prune.misclass(tree.carseats,best=9) #pruned tree
plot(prune.carseats)
text(prune.carseats,pretty=0)
```

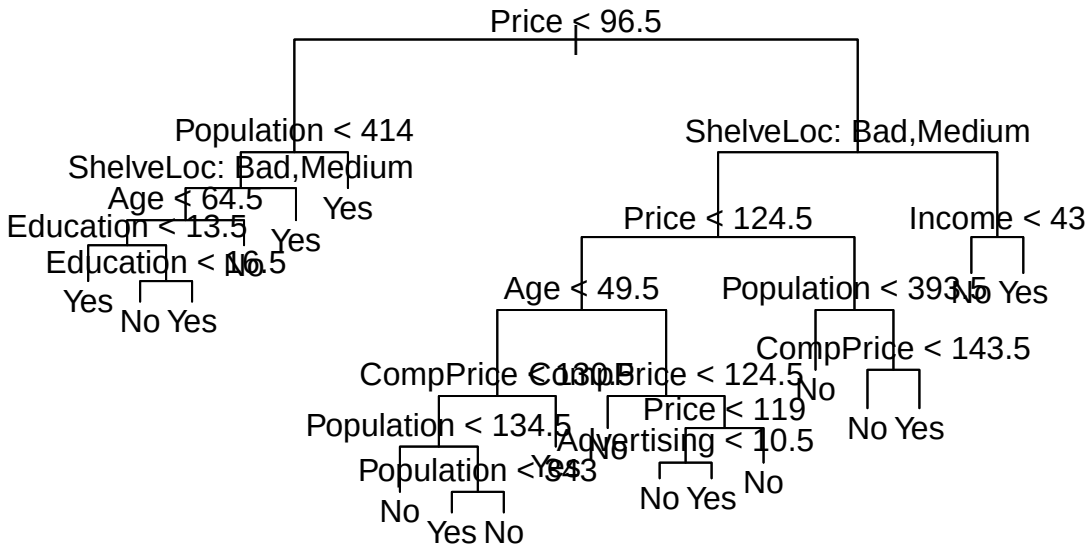


Use the pruned tree to predict the categories for the test data:

```
tree.pred=predict(prune.carseats,Carseats.test,type="class") #prediction
table(tree.pred,High.test)
```

```
##          High.test
## tree.pred No Yes
##       No  97  25
##       Yes 20  58
```

```
prune.carseats=prune.misclass(tree.carseats,best=15)
plot(prune.carseats); text(prune.carseats,pretty=0)
```




```
tree.pred=predict(prune.carseats,Carseats.test,type="class")
table(tree.pred,High.test)
```

```
##          High.test
## tree.pred  No Yes
##       No  102  30
##       Yes  15  53
```

produces a more interpretable tree and improves a prediction rate.

Building the tree

There are numerous algorithms for building trees. The basic steps are

1. Start with all variables at the root node.
2. Try all variables and select the one that best separates the classes.
3. Divide the data according to the rule determined in step 2.
4. Repeat 2,3 recursively until a stopping criterion is reached.
5. Apply class labels to the leaf nodes by majority vote.

Criterion for selecting the best variable to split on

The obvious criterion for choosing the best variable to split on is classification error rate. Since class is predicted by majority vote, this means that the classification error rate is simply the proportion of training observations in the split region that are not of the majority class. The error rate E_{mn} when in region m of the sample space (where regions correspond to tree splits) is

$$E_m = 1 - \max_k(\hat{\rho}_{mk})$$

Where $\hat{\rho}_{km}$ is the proportion of observations in the region m that belong to class k. We then calculate E_m across regions. The split that produces the lowest E_m's is the best one.

However, it turns out that the error rate is not sufficiently sensitive for tree building. Instead, a commonly used criterion is the **Ginni index**:

$$G_m = \sum_{k=1}^{K} \hat{\rho}_{mk}(1 - \hat{\rho}_{mk}) = 1 - \sum_{k=1}^{K} \hat{\rho}_{mk}^2$$

This has its maximum when $\hat{\rho}_{ik} = 0.5$ and minimum when $\hat{\rho}_{ik}$ is one or zero.

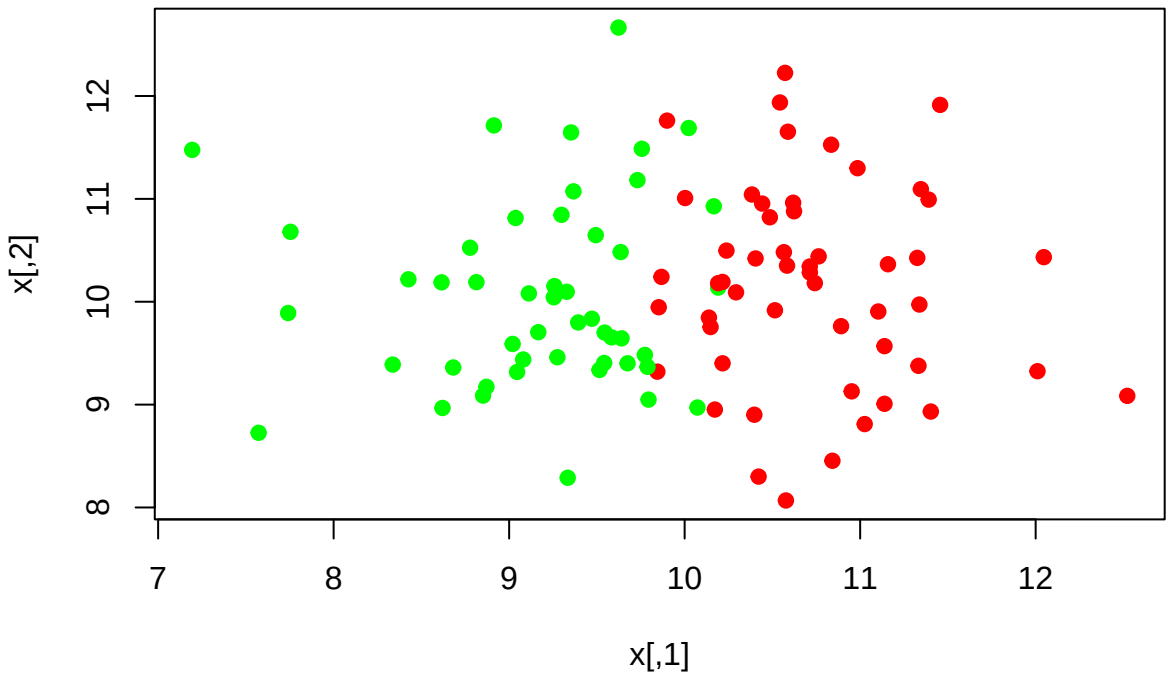
The Gini index for a split is the weighted average of the two regions produced by the splits:

$$G_{split} = \frac{n_1}{n} G_1 + \frac{n2}{n} G_2$$

n_1 and n_2 are the number of points on each side of the spit, $n = n_1 + n_2$, and G_1 and G_2 are the Gini indexes for the regions created by the split.

Demonstrate with some data:

```
set.seed((299))
library(MASS)
n=100
x=mvrnorm(n,mu=c(10,10),Sigma=matrix(c(1,0,0,1),nrow=2))
class=ifelse(x[,1]+rnorm(n,sd=0.25)>10,1,2)

plot(x,col=ifelse(class==1,"red","green"),pch=19)
```



Split on `x1` Take the split on $x1$ as $x1 > 10$. Then calculate the ρ_{mk} values. We will take the region to the left of $x1 = 10$ as $m = 1$ and the region to the right as $m = 2$.

Then, we get

```
(p11=sum(class[x[,1]<=10]==1)/sum(x[,1]<=10)) #proportion of class 1 in region 1
```

```
## [1] 0.08333333
```

```
(p21=sum(class[x[,1]<=10]==2)/sum(x[,1]<=10)) #proportion of class 2 in region 1
```

```
## [1] 0.9166667
```

```
(p12=sum(class[x[,1]>10]==1)/sum(x[,1]>10)) #proportion of class 1 in region 2
```

```
## [1] 0.9230769
```

```
(p22=sum(class[x[,1]>10]==2)/sum(x[,1]>10)) #proportion of class 2 in region 2
```

```
## [1] 0.07692308
```

Note that this split produces high proportions of class 1 in region 2 and high proportions of class 2 in region 1.

The Gini indexes are

```
(G1=1-p11^2-p12^2)
```

```
## [1] 0.1409845
```

```
(G2=1-p12^2-p22^2)
```

```
## [1] 0.1420118
```

The overall Gini index is

```
n1=sum(x[,1]<10)
n2=n-n1

(GSX1=n1/n*G1+n2/n*G2)
```

```
## [1] 0.1415187
```

Split on `x2` Next, we do the same calculation, by splitting on $x2 > 10$. The only difference in the code is that we use `x[,2]` rather than `x[,1]`.

```
(p12=sum(class[x[,2]>10]==1)/sum(x[,2]>10)) #proportion of class 1 in region 2
```

```
## [1] 0.5660377
```

```
(p22=sum(class[x[,2]>10]==2)/sum(x[,2]>10)) #proportion of class 2 in region 2
```

```
## [1] 0.4339623
```

```
(p11=sum(class[x[,2]<=10]==1)/sum(x[,2]<=10)) #proportion of class 1 in region 1
```

```
## [1] 0.4680851
```

```
(p21=sum(class[x[,2]<=10]==2)/sum(x[,2]<=10)) #proportion of class 2 in region 1
```

```
## [1] 0.5319149
```

The second variable has no relationship with class and so the split on `x2` produces class proportions near 0.5.

The Gini indexes are

```
(G1=1-p11^2-p12^2)
```

```
## [1] 0.4604976
```

```
(G2=1-p12^2-p22^2)
```

```
## [1] 0.491278
```

The overall Gini index is

```
n1=sum(x[,2]<10)
n2=n-n1

(GSX2=n1/n*G1+n2/n*G2)
```

```
## [1] 0.4768112
```

We see that the Gini index is much higher (0.4768) for the second split than the first one (0.1415). This is because `x1>10` is an effective split. It produces regions with high proportions of each class. In contrast, class is indendent of `x2` and so `x2>10` is not an effective split and produces regions with approximately the same class proportions.

Algorithm for choosing the split

In the above example, I chose the split point `x1>10` because I had simulated the data and knew that this was the true boundary in the data. How do we determine the split point when we don't know the true boundary?

Here is the basic algorithm:

loop x over remaining predictors
{ loop c over values of split point for x
{ Calculate and store the Gini index for two regions made by (x,c)
** }**
}
optimal (x,c) is the one that produces the smallest Ginni index. Take x as the variable and c as the boundary for the spit for that variable.

A common choice for the sequence of values to check at the split points is the midpoints between the values of variable `x` for each pair of data points.

Example

Let's do an example using the simulated data from the beginning of these notes. First, we need to automate the calculation of the Gini index.

Exercise #1 Write a function `calcGini(dat,x,y,b)` that takes input of a data frame `dat` with a set of predictors and response. The input `x` should specify the column index for the predictor to be tested for a split, the input `y` should specify the column index for the class variable, and the input `b` should specify the boundary point for the split on the the variable specified by `x`. The function should calculate the overall Gini index for the two regions created by the split.

Exercise #2 write a function `bvals(z)` that take input of a vector `z`, sorts `z` and returns a vector that consists of the midpoints between each consectutive pair of sorted values.

Exercise #3 Write a function `find_split(dat,predcol,y)` that takes input of a data frame `dat`, a vector of column indexes in `dat` to be considered as predictors for the split, and a column index `y` for the column with the class. It should implement the algorithm above for finding the split: loop over the values in `predcol`, within those values loop over values from the boundary as generated by your `bvals` function, and calculate the Gini index for each combination of predictor column and boundary. The best split is the one corresponding to the minimum Gini value. Output a list with the best variable to split on and the optimal boundary for the split.

Exercise 4: Make a simulated data set similar to the one at the beginning of these notes, but with more variables. Include some noise variables and some weaker and stronger predictors for class. Test your `find_split` function on this data.